

Documentazione dei file Helm del chart "demo-microservices"

Questo documento descrive la struttura e il comportamento dei file del chart Helm, includendo **Chart.yaml**, **values.yaml** e i template **deployment.yaml**, **service.yaml**, **ingress.yaml**.

1) Chart.yaml

- **apiVersion:** v2 — versione dello schema Chart (Helm 3).
- **name:** demo-microservices — nome del chart.
- **description:** Helm chart con ingress — descrizione sintetica.
- **type:** application — indica che il chart rilascia una applicazione (non una libreria).
- **version:** 0.2.0 — versione del chart (gestione del pacchetto).
- **appVersion:** "1.0" — versione dell'app distribuita (visibile come etichetta/info).

2) values.yaml

Il file **values.yaml** definisce i valori predefiniti usati dai template. I principali blocchi sono:

- **global.imagePullPolicy:** IfNotPresent — policy di pull delle immagini per tutti i container.
- **global.eurekaUrl:** URL del registry Eureka reso disponibile dal servizio demo-eureka-service.
- **ingress.enabled:** true — abilita la creazione della risorsa Ingress.
- **ingress.className:** nginx — classe dell'ingress controller.
- **ingress.hosts:** mappa degli host e dei path; nell'esempio host *demo.local* con path */* che instrada verso *demo-gateway-service* sulla porta 8765.
- **services:** definizione parametrica dei microservizi: demo-eureka-service, demo-service-a, demo-service-b, demo-gateway-service. Per ciascuno: *image*, *port*, *replicaCount*, *servicePort*, *env*, *healthPath*.

3) templates/deployment.yaml

Questo template crea un **Deployment** per ciascun servizio definito in *.Values.services*. È utilizzato un ciclo *range* per iterare sui servizi, impostando metadati, repliche, selettori e il container con immagine, porta, policy di pull e variabili di ambiente opzionali. La *readinessProbe* fa HTTP GET su *healthPath* e sulla porta applicativa, con 10s di ritardo iniziale e periodo di 5s.

```
{{- range $name, $svc := .Values.services }}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ $name }}
spec:
  replicas: {{ $svc.replicaCount }}
  selector:
    matchLabels:
      app: {{ $name }}
  template:
    metadata:
      labels:
        app: {{ $name }}
```

```

spec:
  containers:
    - name: {{ $name }}
      image: {{ $svc.image }}
      imagePullPolicy: {{ $.Values.global.imagePullPolicy }}
      ports:
        - containerPort: {{ $svc.port }}
      env:
{{- if $svc.env }}
{{- toYaml $svc.env | nindent 12 }}
{{- end }}
      readinessProbe:
        httpGet:
          path: {{ $svc.healthPath }}
          port: {{ $svc.port }}
          initialDelaySeconds: 10
          periodSeconds: 5
---
{{- end }}

```

4) templates/service.yaml

Questo template crea un **Service** di tipo *ClusterIP* per ciascun microservizio. Seleziona i pod tramite l'etichetta *app*: *<nome-servizio>* e pubblica una porta logica (*servicePort*) che indirizza alla *targetPort* (porta del container).

```

{{- range $name, $svc := .Values.services }}
apiVersion: v1
kind: Service
metadata:
  name: {{ $name }}
spec:
  selector:
    app: {{ $name }}
  ports:
    - port: {{ $svc.servicePort }}
      targetPort: {{ $svc.port }}
      protocol: TCP
      name: http
      type: ClusterIP
---
{{- end }}

```

5) templates/ingress.yaml

Il template genera un oggetto **Ingress** solo se *.Values.ingress.enabled* è true. Usa la classe specificata in *ingress.className* e definisce regole per ciascun host elencato in *ingress.hosts*; per ogni path viene configurato il backend con *service* e *port*.

```

{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: demo-ingress
  annotations:
    kubernetes.io/ingress.class: {{ .Values.ingress.className }}
spec:

```

```

rules:
{{- range .Values.ingress.hosts }}
  - host: {{ .host }}
    http:
      paths:
        {{- range .paths }}
          - path: {{ .path }}
            pathType: Prefix
            backend:
              service:
                name: {{ .service }}
                port:
                  number: {{ .port }}
        {{- end }}
      {{- end }}
    {{- end }}

```

6) Flusso di rilascio e relazioni

- Per ogni voce in *services* vengono creati 1 Deployment e 1 Service omonimi, allineati tramite l'etichetta *app*.
- Il Gateway e i servizi A/B usano Eureka tramite la variabile d'ambiente *EUREKA_CLIENT_SERVICEURL_DEFAULTZONE* impostata al valore globale *global.eurekaUrl*.
- L'Ingress, quando abilitato, espone pubblicamente il *demo-gateway-service* (o altri target definiti in *ingress.hosts*) sugli host e path configurati.